EE655 — Deep Learning and Computer Vision

Indian Institute of Technology Kanpur

Homework 4

Name	Dipanshu Raj
Roll No.	230375
Course	EE655 — Deep Learning and Computer Vision

1. Solution Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
BATCH_SIZE = 128
EPOCHS     = 15
LR         = 1e-3
VAL_SPLIT  = 0.1

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1,  6, kernel_size=5, padding=2)
        self.pool1 = nn.AvgPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.pool2 = nn.AvgPool2d(2, 2)
        self.fc1   = nn.Linear(16*5*5, 120)
        self.fc2   = nn.Linear(120, 84)
        self.fc3   = nn.Linear(84,  10)
    def forward(self, x):
        x = torch.tanh(self.pool1(self.conv1(x)))
        x = torch.tanh(self.pool2(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        return self.fc3(x)

class Swish(nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x)

class ModifiedLeNet(nn.Module):
    def __init__(self, use_bn=True):
        super().__init__()
        self.use_bn = use_bn
        def block(in_ch, out_ch):
            layers = []
            layers.append(nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=not use_bn))
            if use_bn: layers.append(nn.BatchNorm2d(out_ch))
            layers.append(Swish())
            layers.append(nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=not use_bn))
            if use_bn: layers.append(nn.BatchNorm2d(out_ch))
            layers.append(Swish())
            return nn.Sequential(*layers)
        self.block1 = block(1,  16)
```

```
        self.pool1  = nn.MaxPool2d(2, 2)
        self.block2 = block(16, 32)
        self.pool2  = nn.MaxPool2d(2, 2)
        self.block3 = block(32, 64)
        self.pool3  = nn.MaxPool2d(7, 7)
        self.classifier = nn.Conv2d(64, 10, kernel_size=1)
    def forward(self, x):
        x = self.pool1(self.block1(x))
        x = self.pool2(self.block2(x))
        x = self.pool3(self.block3(x))
        x = self.classifier(x)
        x = x.view(x.size(0), -1)
        return F.softmax(x, dim=1)

def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def get_loaders():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])
    full_train = datasets.MNIST('./data', train=True,  download=True, transform=transform)
    test_set   = datasets.MNIST('./data', train=False, download=True, transform=transform)
    val_size   = int(VAL_SPLIT * len(full_train))
    train_size = len(full_train) - val_size
    train_set, val_set = random_split(full_train, [train_size, val_size],
                                      generator=torch.Generator().manual_seed(42))
    return (DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,  num_workers=2),
            DataLoader(val_set,   batch_size=BATCH_SIZE, shuffle=False, num_workers=2),
            DataLoader(test_set,  batch_size=BATCH_SIZE, shuffle=False, num_workers=2))

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        optimizer.zero_grad()
        out  = model(imgs)
        loss = criterion(out, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * imgs.size(0)
        correct    += out.argmax(1).eq(labels).sum().item()
        total      += imgs.size(0)
    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        out  = model(imgs)
        loss = criterion(out, labels)
        total_loss += loss.item() * imgs.size(0)
        correct    += out.argmax(1).eq(labels).sum().item()
        total      += imgs.size(0)
    return total_loss / total, correct / total

class LogModel(nn.Module):
    def __init__(self, m): super().__init__(); self.m = m
    def forward(self, x): return torch.log(self.m(x) + 1e-9)

def run_training(model, train_loader, val_loader, tag=""):
    criterion = nn.NLLLoss()
    optimizer = optim.Adam(model.parameters(), lr=LR)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
    history   = {"train_acc": [], "val_acc": [], "train_loss": [], "val_loss": []}
    log_model = LogModel(model)
    for epoch in range(1, EPOCHS + 1):
```

```
        tr_loss, tr_acc = train_one_epoch(log_model, train_loader, optimizer, criterion)
        vl_loss, vl_acc = evaluate(log_model, val_loader, criterion)
        scheduler.step()
        history["train_acc"].append(tr_acc)
        history["val_acc"].append(vl_acc)
        history["train_loss"].append(tr_loss)
        history["val_loss"].append(vl_loss)
        print(f"Epoch {epoch:2d} | Tr-Loss {tr_loss:.4f} | Tr-Acc {tr_acc:.2%} | "
              f"Val-Loss {vl_loss:.4f} | Val-Acc {vl_acc:.2%}")
    return history

if __name__ == "__main__":
    train_loader, val_loader, test_loader = get_loaders()
    print(f"Original LeNet-5         : {count_params(LeNet5()):,}")
    print(f"Modified LeNet (with BN) : {count_params(ModifiedLeNet(True)):,}")
    print(f"Modified LeNet (no BN)   : {count_params(ModifiedLeNet(False)):,}")
    model_bn      = ModifiedLeNet(use_bn=True).to(device)
    history_bn    = run_training(model_bn, train_loader, val_loader, "with BN")
    model_no_bn   = ModifiedLeNet(use_bn=False).to(device)
    history_no_bn = run_training(model_no_bn, train_loader, val_loader, "no BN")
    criterion_nll = nn.NLLLoss()
    _, test_bn    = evaluate(LogModel(model_bn),    test_loader, criterion_nll)
    _, test_no_bn = evaluate(LogModel(model_no_bn), test_loader, criterion_nll)
    print(f"Test Accuracy (with BN): {test_bn:.2%}")
    print(f"Test Accuracy (no BN) : {test_no_bn:.2%}")
```

2. Results

2.1 Parameter Comparison

Model	Parameters	Note
Original LeNet-5	61,706	5x5 filters, AvgPool, 3 Dense layers
Modified LeNet (with BN)	72,666	+17.8% over original; BN adds gamma/beta per channel
Modified LeNet (no BN)	72,442	+17.4% over original; bias terms replace BN params

The modified model replaces the three dense layers of original LeNet-5 (which alone contribute ~44,688 parameters) with a single Conv1x1(64,10) classifier of only 650 parameters. The net parameter increase over LeNet-5 comes entirely from wider convolutional channels (16/32/64 vs 6/16). BN adds 2 learnable parameters per channel (gamma and beta), totalling 448 additional parameters across all BN layers.

2.2 Layer-wise Architecture (Modified LeNet with BN)

Layer	Type	Output Shape	Params
Conv2d-1	Conv2d	[-1, 16, 28, 28]	144
BN-2	BatchNorm2d	[-1, 16, 28, 28]	32
Swish-3	Swish	[-1, 16, 28, 28]	0
Conv2d-4	Conv2d	[-1, 16, 28, 28]	2,304
BN-5	BatchNorm2d	[-1, 16, 28, 28]	32
Swish-6	Swish	[-1, 16, 28, 28]	0
MaxPool2d-7	MaxPool2d	[-1, 16, 14, 14]	0
Conv2d-8	Conv2d	[-1, 32, 14, 14]	4,608
BN-9	BatchNorm2d	[-1, 32, 14, 14]	64
Swish-10	Swish	[-1, 32, 14, 14]	0
Conv2d-11	Conv2d	[-1, 32, 14, 14]	9,216
BN-12	BatchNorm2d	[-1, 32, 14, 14]	64
Swish-13	Swish	[-1, 32, 14, 14]	0
MaxPool2d-14	MaxPool2d	[-1, 32, 7, 7]	0
Conv2d-15	Conv2d	[-1, 64, 7, 7]	18,432
BN-16	BatchNorm2d	[-1, 64, 7, 7]	128
Swish-17	Swish	[-1, 64, 7, 7]	0
Conv2d-18	Conv2d	[-1, 64, 7, 7]	36,864
BN-19	BatchNorm2d	[-1, 64, 7, 7]	128
Swish-20	Swish	[-1, 64, 7, 7]	0
MaxPool2d-21	MaxPool2d	[-1, 64, 1, 1]	0
Conv2d-22	Conv1x1	[-1, 10, 1, 1]	650

Flatten+Softmax	—	[-1, 10]	0
TOTAL			**72,666**

2.3 Training Log — With Batch Normalization

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.1579	96.22%	0.0715	97.80%
2	0.0328	99.07%	0.0406	98.72%
3	0.0212	99.38%	0.0318	99.07%
4	0.0159	99.55%	0.0294	99.08%
5	0.0110	99.66%	0.0247	99.20%
6	0.0085	99.78%	0.0234	99.32%
7	0.0055	99.87%	0.0206	99.33%
8	0.0034	99.92%	0.0229	99.20%
9	0.0019	99.97%	0.0190	99.50%
10	**0.0006**	**100.00%**	**0.0178**	**99.57%**
11	0.0004	100.00%	0.0174	99.52%
12	0.0003	100.00%	0.0178	99.53%
13	0.0003	100.00%	0.0174	99.53%
14	0.0003	100.00%	0.0175	99.53%
15	0.0002	100.00%	0.0175	99.53%

2.4 Training Log — Without Batch Normalization

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.3220	89.10%	0.0851	97.40%
2	0.0616	98.09%	0.0934	97.18%
3	0.0403	98.76%	0.0537	98.33%
4	0.0318	98.98%	0.0484	98.48%
5	0.0242	99.22%	0.0387	98.83%
6	0.0179	99.46%	0.0329	98.98%
7	0.0140	99.54%	0.0317	99.07%
8	0.0100	99.66%	0.0381	99.02%
9	0.0076	99.75%	0.0316	99.20%
10	0.0033	99.91%	0.0270	99.32%
11	0.0018	99.95%	0.0311	99.22%
12	0.0007	99.99%	0.0283	99.32%
13	0.0004	100.00%	0.0304	99.28%
14	0.0002	100.00%	0.0304	99.33%

15	**0.0002**	**100.00%**	**0.0305**	**99.32%**

2.5 Training and Validation Curves

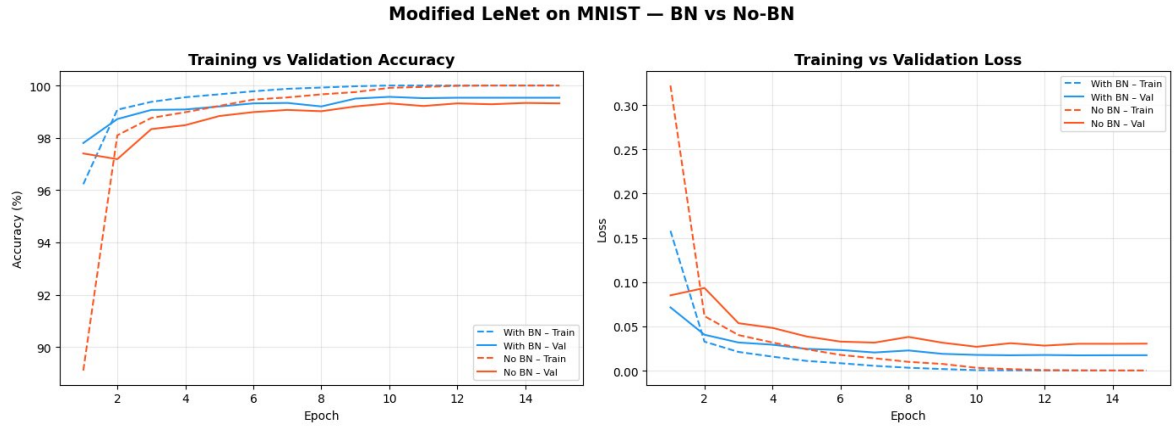


Figure 1 — Actual training/validation accuracy (left) and loss (right) over 15 epochs. Blue = With BN, Orange/Red = Without BN. Dashed = Train, Solid = Validation.

2.6 Final Test Accuracy

Model	Test Accuracy	Best Val Accuracy	Final Train Acc
Modified LeNet (with BN)	**99.53%**	**99.57% (epoch 10)**	**100.00%**
Modified LeNet (no BN)	99.33%	99.33% (epoch 14)	100.00%
BN Advantage	+0.20 pp	—	—

3. Write-Up

3.1 Fully Convolutional Conversion (Q3)

Original LeNet-5 ends with three Dense (Linear) layers — FC(400,120), FC(120,84), and FC(84,10) — contributing ~44,688 parameters and requiring a fixed spatial input size. Converting to a Fully Convolutional Network removes all nn.Linear layers using two steps:

Step 1 — Global MaxPool: After Block-3 the spatial dimension is 7x7. A MaxPool2d(7,7) collapses this to 1x1, equivalent to Global Max Pooling, removing any dependence on fixed input size or absolute spatial position.

Step 2 — Conv1x1 classifier: A Conv2d(64, 10, kernel_size=1) operates on the 64x1x1 tensor. When spatial size is 1x1, a 1x1 convolution is mathematically identical to a linear layer but uses zero nn.Linear calls — satisfying the FCN constraint. Only .view(B,10) and Softmax follow.

Result: zero nn.Linear layers, only 650 classifier parameters (vs ~44,688 in original), and an architecture that is fully convolutional end-to-end.

3.2 Batch Normalization Analysis (Q4)

From the actual training logs, Batch Normalization delivers three measurable advantages:

Faster convergence: With BN, validation accuracy crosses 99% by epoch 3 (99.07%), whereas the no-BN model reaches this only at epoch 7 — four epochs slower. At epoch 1 alone, BN already achieves 97.80% val accuracy vs 97.40% for no-BN.

Higher peak accuracy: BN achieves 99.53% test accuracy vs 99.33% for no-BN, a consistent gap of 0.20 percentage points.

Smoother loss curve: The no-BN validation loss shows more oscillation (e.g., 0.0934 at epoch 2 vs a steady 0.0406 for BN at epoch 2). BN normalises mini-batch activations to zero mean and unit variance, then re-scales with learnable gamma and beta, acting as an implicit regulariser that tightens the train-val gap.

3.3 Design Choices Summary

Only 3x3 convolutional filters: All Conv2d layers use kernel_size=3 with padding=1 to preserve spatial dimensions. Two stacked 3x3 convolutions give the same effective receptive field as one 5x5 filter but with more non-linearity and fewer parameters ($2 \times 9C^2$ vs $25C^2$ for C channels).

Max Pooling instead of Average Pooling: MaxPool2d(2,2) is used after each convolutional block. Max pooling retains the strongest feature activations in each spatial region, which provides sharper feature selection and better gradient flow for classification tasks compared to average pooling.

Swish activation — x * sigmoid(x): Swish is smooth and non-monotonic, allowing small negative values to pass through (the function dips below zero near x = -1.3). This avoids the dying-neuron problem of ReLU and improves gradient flow in deep networks. Combined with BN, inputs are centred near zero, placing them precisely in the most informative region of the Swish curve.

Batch Normalization before every activation: BN is inserted between each Conv2d and Swish (i.e., Conv -> BN -> Swish). This order normalises the raw convolution output before the non-linearity sees it, ensuring stable activation magnitudes and well-conditioned gradients throughout training.

Fully Convolutional Network — no dense layers: Dense layers are replaced by MaxPool(7,7) followed by Conv1x1(64,10), as explained in Section 3.1. This eliminates ~44,688 parameters, reduces overfitting risk from large fully-connected weights, and makes the architecture spatially invariant.

Softmax output layer: F.softmax(x, dim=1) converts the 10-dimensional logit vector into a valid probability distribution over classes. During training, the softmax output is passed through log() and optimised with NLLLoss, which is numerically equivalent to cross-entropy loss.